

PeakCheck — Anleitung

Interaktives Multippeak-Fitting und Peak-Präsenz-Prüfung für beliebige x/y-Daten

Lukas Knauer

RPTU Kaiserslautern-Landau · AG Schünemann

Version 1.0.0 · MIT-Lizenz · 2026

Was macht PeakCheck? Es bestimmt Peaks in einem Referenzdatensatz (z. B. einem aufsummierten Spektrum) und prüft anschließend, ob dieselben Peaks in einem oder mehreren Komponenten-Datensätzen (z. B. Subspektren) vorhanden sind. Es funktioniert mit beliebigen 2- oder 3-spaltigen x/y-Daten — Spektren, Beugungsmuster, Chromatogramme, Messreihen.

PeakCheck — Ablauf einer Auswertung



Referenzdatensatz definiert die Peaks · Komponenten werden dagegen geprüft · eine TOML-Config steuert alles

Abbildung 1. Ablauf einer Auswertung: vom Einlesen über Suche und Fit bis zum Vergleich und zur Ausgabe.

1 Installation & Start

PeakCheck ist ein kleines Python-Paket mit einem rückwärtskompatiblen Starter (`peakcheck.py`). Benötigt werden Python 3.10+ sowie die Pakete `numpy`, `scipy`, `matplotlib`, `openpyxl` und `Tk` für die grafische Oberfläche.

```
# Pakete installieren (einmalig)
pip install numpy scipy matplotlib openpyxl
# Tk ist bei Windows/macOS schon dabei; unter Linux:
sudo apt install python3-tk
# Fuer Python < 3.11 zusaetzlich der TOML-Leser:
pip install tomli
```

Es gibt drei gleichwertige Wege, das Programm zu starten — nimm den, der zu deiner Arbeitsweise passt:

```
# 1) Grafisch (Standard): Fenster oeffnen, einstellen, klicken, fitten
python -m peakcheck           # oder einfach: peakcheck (nach pip install)
python peakcheck.py           # funktioniert weiterhin als Starter

# 2) Ohne Fenster, gesteuert ueber eine TOML-Datei (Batch/HPC)
python -m peakcheck --config job.toml --no-gui

# 3) Eine kommentierte Konfigurationsvorlage schreiben
python -m peakcheck --write-template job.toml
```

In Spyder am besten als eigenständiges Programm starten (nicht in der IPython-Konsole), damit sich ein echtes Fenster öffnet. Alternativ in der Konsole vorher `%matplotlib qt5` setzen.

2 Eingabedaten

PeakCheck liest spaltenweise Textdaten unabhängig von der Dateiendung: `.dat`, `.txt`, `.csv`, `.xy`, `.asc`, `.tsv`, `.prn`, `.nis` und ähnliche funktionieren, solange die Datei zwei oder drei numerische Spalten enthält. Werte dürfen durch Leerzeichen, Tabulatoren, Kommas oder Semikola getrennt sein; Kopfzeilen, Kommentarzeichen (`## ! \%`) und ein deutsches Dezimalkomma (z. B. `10,5`) werden automatisch erkannt und behandelt. (Einzig `.fio` wird gesondert behandelt; siehe unten.)

Format	Bedeutung	Auswirkung auf den Fit
<code>x y</code>	zwei Spalten, keine Fehler	ungewichteter Fit; Rauschen wird aus den Daten geschätzt
<code>x y y_fehler</code>	drei Spalten mit Messfehler	gewichteter Fit; Fehler gehen als $1/\sigma$ in den Fit ein

FIO-Dateien (DESY/PETRA III). Dateien mit der Endung `.fio` werden direkt gelesen: PeakCheck wertet den `%d`-Datenblock mit seinen `Col`-Spaltendeklarationen aus (die `%c/%p`-Kommentar- und Parameterblöcke werden ignoriert). Da solche Dateien viele Spalten enthalten, wählen zwei Einstellungen die gewünschten aus: `fio_x_column` (Standard: erste Spalte, meist die Energieachse) und `fio_y_column` (Standard `"nisp"`, das NIS-Signal). Beides kann ein Spaltenname (z. B. `"nfsp"`) oder ein 1-basierter Index sein. FIO-Dateien haben keine Fehlerspalte, daher wird das Rauschen geschätzt, sofern `error_column` nicht auf eine bestimmte Spalte zeigt. `<name>_*.<endung>`. Liegt die Referenz also in `probe.txt`, werden `probe_01.txt`, `probe_42.txt` usw. als Komponenten erkannt. Das Muster ist über `component_glob` frei einstellbar.

3 Die grafische Oberfläche

Bedienschritte

1. **Ordner öffnen** über *Open folder*. ... Es erscheint ein Dialog mit allen Datendateien des Ordners. Markiere eine als Referenz (Radio-Knopf in der Spalte „Ref“) und beliebig viele als Komponenten (Häkchen in der Spalte „Cmp“). Mit *OK* bestätigen — die Daten erscheinen sofort zusammen mit der untergrundkorrigierten Kurve (blau).
2. **Suchbereich** über `x_min/x_max` eingrenzen (mit *Enter* bestätigen); der aktive Bereich wird orange hinterlegt.
3. **Profil und Breiten** wählen: `voigt`, `gauss` oder `lorentz` sowie die Halbwertsbreiten `WG` (Gauß) und `WL` (Lorentz). Die resultierende effektive FWHM des gewählten Profils wird daneben als nicht editierbarer Wert angezeigt (sie aktualisiert sich beim Bearbeiten von `WG/WL` oder des Profils).
4. **Peaks finden**: mit den vier Slidern (Prominenz, Mindestabstand, Mindest-SNR, Untergrundfenster) die automatische Suche steuern; was jeder einzelne bewirkt, steht unten unter „Parameter wählen & gute Praxis“.
5. **Basislinie und Verfeinerung** (vierte Zeile, optional): die `baseline_method` für den Fit wählen (`rolling_min`, `polynomial` oder `als`, daneben Polynomgrad und `ALS- $\lambda/p$`) und *Refine positions* aktivieren, um die Lagen nach dem Fit innerhalb von `refine window` zu verfeinern. Die Standardwerte reproduzieren das bisherige Verhalten.
6. **Feinkorrektur per Maus** direkt im Plot:
 - *Linksklick* — fügt am nächstgelegenen Datenpunkt einen Peak hinzu

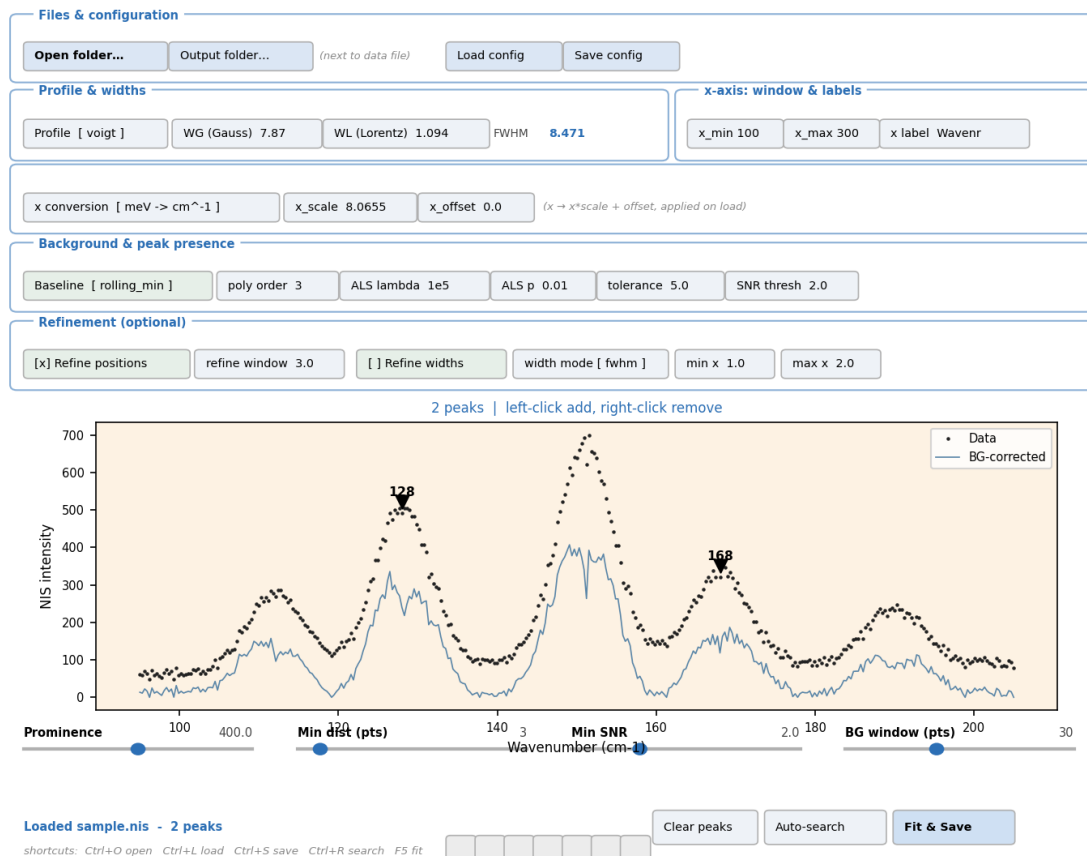


Abbildung 2. Das PeakCheck-Fenster: die Bedienelemente sind in beschriftete Gruppen gefasst (Dateien & Konfiguration; Profil & Breiten; x-Achse: Fenster, Beschriftungen & Umrechnung; Untergrund & Peak-Präsenz; Verfeinerung), gefolgt vom interaktiven Plot mit Peak-Markern, den Such-Slidern und dem Knopf *Fit & Save*. Die Tastaturkürzel stehen unten. Felder, die zur aktuellen Auswahl nicht passen (z. B. die ALS-Parameter, wenn die Basislinie nicht ALS ist), sind ausgegraut.

- *Rechtsklick* — entfernt den nächstgelegenen Peak-Marker

7. **Fit & Save** startet den Fit, die Präsenzprüfung in den Komponenten und schreibt alle Ausgabedateien.

Tipp zur Reihenfolge: erst den Suchbereich setzen, dann das Untergrundfenster so justieren, dass die blaue Kurve die Peaks sauber abbildet, anschließend Mindest-SNR (≈ 2) und Prominenz feinjustieren — und zuletzt per Mausklick einzelne Peaks ergänzen oder entfernen.

Über *Save config* bzw. *Load config* lassen sich alle Einstellungen als TOML-Datei sichern und wieder laden — so ist jede Auswertung reproduzierbar.

3.1 Einheiten der x-Achse umrechnen

In der dritten Bedienzeile lässt sich die x-Achse beim Einlesen umrechnen. Das Auswahlfeld *x conversion* bietet gängige Umrechnungen als Vorlage; sie setzt *x_scale*, *x_offset* sowie Achsenbeschriftung und Einheit automatisch. Für NIS/NRVS wählst du „meV $\rightarrow$ cm⁻¹“ — danach läuft die gesamte Auswertung in cm⁻¹.

Die Umrechnung ist affin ($x \rightarrow x \cdot x_scale + x_offset$), du kannst also über die Felder *x_scale*/*x_offset* auch einen eigenen Faktor eingeben. Reziproke Umrechnungen wie Wellenlänge $\leftrightarrow$ Wellenzahl (cm⁻¹ = $10^7/\lambda$ [nm]) sind nicht linear und nur über die TOML-Datei erreichbar (siehe §4).

Vorlage	Faktor (x_scale)	typische Anwendung
meV $\rightarrow$ cm ⁻¹	8,065544	NIS/NRVS, pDOS
cm ⁻¹ $\rightarrow$ meV	0,12398419	Rückrichtung
meV $\rightarrow$ THz	0,24179893	THz-/inelastische Spektroskopie
cm ⁻¹ $\rightarrow$ THz	0,029979246	FIR, Phononen
eV $\rightarrow$ cm ⁻¹	8065,544	allgemeine Energieachsen
meV $\rightarrow$ K (k_B)	11,604518	thermische Skala

Wichtig: Die Umrechnung wird beim Einlesen angewandt. Alle anderen Werte in x-Einheiten — `x_min`, `x_max`, `wg`, `wl`, `tolerance` — beziehen sich daher auf die bereits umgerechneten Werte (also z. B. cm⁻¹, nicht meV).

4 Kommandozeile

Neben der GUI besitzt PeakCheck eine kleine Kommandozeile (das Konsolenskript `peakcheck`, gleichbedeutend `python -m peakcheck`):

Befehl	Wirkung
<code>peakcheck</code>	startet die grafische Oberfläche
<code>peakcheck -write-template job.toml</code>	schreibt eine kommentierte Konfigvorlage und beendet sich
<code>peakcheck -config job.toml -no-gui</code>	läuft headless (Batch); benötigt eine Config
<code>peakcheck -config job.toml -validate</code>	prüft eine Config und beendet sich (Exit-Code $\neq$ 0 bei Fehler)
<code>peakcheck -list-conversions</code>	listet die x-Achsen-Presets und beendet sich
<code>peakcheck -reference data.dat</code>	überschreibt die Referenzdatei aus der Config
<code>peakcheck -output-dir DIR</code>	schreibt alle Ausgaben nach DIR (Batch; überschreibt <code>output_dir</code>)
<code>peakcheck -version</code>	gibt die Version aus
<code>peakcheck -debug</code>	zeigt bei Fehlern den vollen Traceback statt einer Kurzmeldung

Ein Headless-Lauf schreibt exakt dieselben Plots, dieselbe Excel-Mappe und dieselbe CSV wie die GUI; eine aus der GUI gespeicherte Config reproduziert ihr Ergebnis also im Batch. `-validate` eignet sich in Skripten, um bei einer fehlerhaften Config sofort abubrechen. Bei einer fehlerhaften Config oder Datendatei gibt der Befehl eine kurze **PeakCheck: ⚠️**-Meldung aus und endet mit einem Fehlercode; `-debug` zeigt den vollen Python-Traceback.

5 Eine erste Analyse

Das Paket bringt ein lauffähiges Beispiel in `example_data/` mit.

Headless, in einer Zeile (aus `example_data/`):

```
python -m peakcheck --config nis_example.toml --no-gui
```

PeakCheck lädt die Referenz `sample.nis`, fittet fünf Peaks, prüft die beiden Komponentendateien `sample_A.nis` und `sample_B.nis` und schreibt die Plots, die Excel-Mappe und die CSV neben die Daten.

Interaktiv: `peakcheck` starten, *Open folder...* (Strg+O) anklicken und `sample.nis` als Referenz, `sample_A.nis` und `sample_B.nis` als Komponenten wählen. Die *x conversion* meV $\rightarrow$ cm⁻¹ ein-

stellen, *Auto-search peaks* (Strg+R) drücken oder die Peaks von Hand anklicken, dann *Fit & Save* (F5); siehe Abbildung 2.

Der Fit reproduziert die Amplituden, $\chi^2_\nu \approx 5,5$ und $R^2 \approx 0,945$ aus dem Supplement und meldet, welche der fünf Peaks jede Komponente enthält.

6 Konfiguration über TOML

Für Batch-Läufe und reproduzierbare Pipelines steuert eine TOML-Datei sämtliche Parameter. Eine kommentierte Vorlage erzeugt `python -m peakcheck -write-template job.toml`. Im Folgenden sind alle verfügbaren Schlüssel aufgelistet — gruppiert nach TOML-Sektion, mit Standardwert, Beschreibung und Hinweis, wann man sie ändert.

Schlüssel	Default	Bedeutung
<code>reference_file</code>	<code>"data.txt"</code>	Pfad zur Referenzdatei, in der die Peak-Positionen bestimmt werden. Ein relativer Pfad wird gegen das Verzeichnis der Config-Datei aufgelöst, sodass Config und Daten portabel bleiben (batch-tauglich); absolute Pfade werden unverändert genutzt. Ein <code>-reference</code> auf der Kommandozeile überschreibt dies und behält die übliche Shell-Semantik (relativ zum Arbeitsverzeichnis).
<code>component_glob</code>	<code>""</code> (leer)	Suchmuster für Komponentendateien. Leer = automatisch <code><stem>_*.<ext></code> (z. B. <code>sample.nis</code> → <code>sample_A.nis</code>). Mit explizitem Muster (z. B. <code>"sub_*.dat"</code>) übersteuerbar; aus der GUI wird die Auswahl direkt im Ordnerdialog übergeben.
<code>error_column</code>	<code>"auto"</code>	<code>"auto"</code> = dritte Spalte als Fehler nutzen, wenn vorhanden, sonst aus den Daten schätzen. <code>"none"</code> = Fehler immer schätzen. Ganze Zahl = expliziter Spaltenindex (0-basiert).
<code>fio_x_column</code>	<code>""</code>	nur FIO: x-Spalte per Name oder 1-basiertem Index; leer = erste Spalte.
<code>fio_y_column</code>	<code>"nisp"</code>	nur FIO: y-Spalte per Name (z. B. <code>"nfsp"</code>) oder 1-basiertem Index.

[input] — Eingabe

Schlüssel	Default	Bedeutung
<code>x_conversion</code>	<code>""</code> (leer)	Benannte Voreinstellung wie <code>"meV -> cm^-1"</code> , <code>"cm^-1 -> THz"</code> oder <code>"nm -> eV (reciprocal)"</code> . Setzt automatisch Scale, Offset, Beschriftung und Einheit.
<code>x_transform</code>	<code>"affine"</code>	<code>"affine"</code> : $x' = x \cdot s + o$ (z. B. Energieeinheiten). <code>"reciprocal"</code> : $x' = s/x + o$ (z. B. Wellenlänge → Wellenzahl).
<code>x_scale, x_offset</code>	<code>1.0, 0.0</code>	Manueller Skalenfaktor und Offset. Wird ignoriert, sobald <code>x_conversion</code> gesetzt ist.
<code>x_label, y_label</code>	<code>"x", "y"</code>	Achsenbeschriftungen für Plots und Ausgabedateien.
<code>x_unit</code>	<code>""</code> (leer)	Einheit der x-Achse; erscheint in Klammern hinter dem Label und in den Excel-Header-Zeilen (Origin-Stil).

[xaxis] — x-Achse

[profile] — Linienprofil und Halbwertsbreiten

[window] — Such- und Fit-Fenster

Schlüssel	Default	Bedeutung
profile	"voigt"	"voigt" (exakte Faltung Gauß⊗Lorentz), "gauss", "lorentz". Voigt ist der physikalisch korrekte Standard für die meisten Spektroskopien.
wg	7.8701	Gauß-FWHM in x-Einheiten — die instrumentelle Auflösung. Bei NIS an BL09XU (SPring-8): $\sim 0,8 \text{ meV} \approx 6,4 \text{ cm}^{-1}$; an P01 (PETRA III): siehe Beamline-Spezifikation.
wl	1.094	Lorentz-FWHM in x-Einheiten — die natürliche Linienbreite plus thermische Beiträge. Bei nicht zerfallenden Phononen üblicherweise klein gegenüber wg.

Schlüssel	Default	Bedeutung
x_min, x_max	"", ""	Untere und obere Grenze des Auswertungsbereichs in x-Einheiten. Leer = offene Grenze (gesamte Datei). Beeinflusst Peak-Suche und Fit; erscheint zur Identifikation im Dateinamen (z. B. _100_200).
bg_window	30	Größe des Fensters für den Untergrund-Abzug (rollendes Minimum, in Punkten). Faustregel: etwas größer als ein Peak, deutlich kleiner als die Untergrund-Krümmung. 0 schaltet den Abzug aus.
baseline_method	rolling_min	Basislinien-Schätzer für Fit, Statistik und Präsenzprüfung: rolling_min (Standard), polynomial oder als (asymmetrische kleinste Quadrate). Die glatten Varianten eignen sich für gekrümmte Untergründe; baseline_poly_order (3), baseline_als_lambda (10^5) und baseline_als_p (0,01) stellen sie ein.

Schlüssel	Default	Bedeutung
prominence_init	200.0	Mindest-Prominenz in y-Einheiten (<code>scipy.signal.find_peaks</code>). Höhere Werte fangen weniger Rauschen, übersehen aber schwache Peaks. In der GUI live per Schieberegler nachstellbar.
distance_init	3	Mindestabstand benachbarter Peaks in Punkten. Verhindert, dass eng beieinander liegende Rauschspitzen als zwei Peaks gezählt werden.
min_snr_init	2.0	SNR-Schwelle für die Auto-Suche (Untergrund-korrigiertes Signal). Erst akzeptiert, wenn die Peak-Höhe $\geq \text{min_snr_init}$ -Fehler beträgt.

[search] — Automatische Peak-Suche

[refine] — optionale Positions- und Breitenverfeinerung Mit `refine = true` werden die Peak-Positionen nach dem Fit durch einen beschränkten nichtlinearen Schritt verfeinert (die Amplituden bleiben nichtnegativ); `refine_window` (Standard 3,0, in x-Einheiten) begrenzt, wie weit sich jede Position verschieben darf. Mit `refine_widths = true` wird zusätzlich eine Breite pro Peak verfeinert: `width_mode` wählt, was sich verbreitert ("fwhm", Standard, skaliert den ganzen Voigt; "sigma" skaliert nur den Gauß-Anteil), und die Breite bleibt zwischen `width_min_factor` und `width_max_factor` mal der instrumentellen Breite (Standard 1,0 .. 2,0, also nie schmaler als das Instrument und höchstens doppelt so breit). Beides ist standardmäßig aus; das Ergebnis ändert sich also nur, wenn man es aktiviert.

[presence] — Präsenzprüfung in den Komponenten

[errors] — Fehlerberichterstattung

[output] — Ausgabe Zusätzlich kann man die Peak-Positionen direkt vorgeben (statt sie interaktiv zu wählen oder von der Auto-Suche zu nehmen):

Schlüssel	Default	Bedeutung
<code>tolerance</code>	5.0	$\pm$ Positionsfenster in x-Einheiten, in dem ein Komponenten-Peak als „dasselbe“ wie ein Referenz-Peak gilt. Sollte größer sein als die typische Schwerpunktsverschiebung zwischen Spektren, kleiner als der Peak-Abstand.
<code>snr_thresh</code>	2.0	SNR-Schwelle, ab der ein Peak in der Komponente als „vorhanden“ gilt. Bei verrauschten Spektren erhöhen (4–6), bei sehr sauberen absenken (2–3).
<code>presence_baseline_corrected</code>	<code>true</code>	<code>true</code> = SNR am Untergrund-korrigierten Signal (Höhe über Basislinie — der physikalisch sinnvolle Wert). <code>false</code> = SNR auf den Rohdaten (enthält Untergrund); selten gewünscht.

Schlüssel	Default	Bedeutung
<code>error_mode</code>	<code>"statistical"</code>	<code>"statistical"</code> = wahre 1σ aus $(B^T W^2 B)^{-1}$, unskaliert (entspricht <code>absolute_sigma=True</code> in <code>scipy.curve_fit</code>). <code>"scaled"</code> = mit $\sqrt{\chi_{\text{red}}^2}$ multipliziert — geeignet, wenn nur relative Fehlergewichtungen bekannt sind.

```
peaks = [112.0, 128.0, 151.0, 168.0, 190.0]
```

Die Liste wird als finale Peak-Menge benutzt — die Auto-Suche entfällt komplett. Das ist der reproduzierbarste Modus für Manuskript-Pipelines.

7 Parameter wählen & gute Praxis

Ein kurzer, praktischer Leitfaden; die Begründung liefert das Supplement.

Breiten (`wg`, `wl`). Das ist die *instrumentelle* Auflösung und wird festgehalten. Aus Instrument/-Beamline setzen, nicht aus den Daten. Ein reduziertes $\chi_{\nu}^2 \gg 1$ bedeutet meist, dass die Breiten nicht zu den Daten passen (oder ein Peak fehlt) — nicht, dass die Amplituden falsch sind.

Untergrund (`baseline_method`). Das voreingestellte rollende Minimum ist robust für schmale Peaks auf breitem Sockel. Bei glatt gekrümmtem Untergrund folgen `polynomial` oder `als` der Kurve treuer; ALS ist meist die beste Allzweckwahl (Abbildung 3). Dieselbe Basislinie nutzen Fit, Statistik und Präsenzprüfung.

Die vier Such-Slider. Die automatische Suche (`scipy.signal.find_peaks` auf dem untergrund-korrigierten Signal) wird von vier Reglern gesteuert, die in der GUI live wirken:

- **Prominenz** (`prominence_init`, y-Einheiten): wie weit ein Peak aus seiner lokalen Umgebung herausragen muss — die Höhe des Gipfels über dem höheren der beiden benachbarten Täler, nicht der absolute y-Wert. Das ist der wichtigste Regler gegen Rauschen: erhöhen, um Rauschspitzen zu verwerfen, senken, um schwache Banden zu fangen.
- **Mindestabstand** (`distance_init`, in Datenpunkten): der kleinste erlaubte Abstand zwischen zwei gefundenen Peaks. Verhindert, dass eine verrauschte Flanke als mehrere dicht stehende Peaks gezählt wird; liegen zwei Kandidaten näher zusammen, gewinnt der prominentere.

Schlüssel	Default	Bedeutung
output_dir	"" (leer)	Zielordner für alle Ausgaben (Plots, Excel, CSV). Leer = neben der Referenzdatei. In der GUI auch per Knopf „Output folder...“ wählbar.
write_csv	true	ASCII-CSV-Datei mit allen Ergebnissectionen schreiben. Auch mit false bleibt das Excel-File erhalten.
write_excel	true	Excel-Workbook schreiben.
plot_dpi	150	Auflösung der gespeicherten PNG-Plots. 150 reicht für Manuskript-Vorschauen; für Druckpublikation 300+ einstellen.

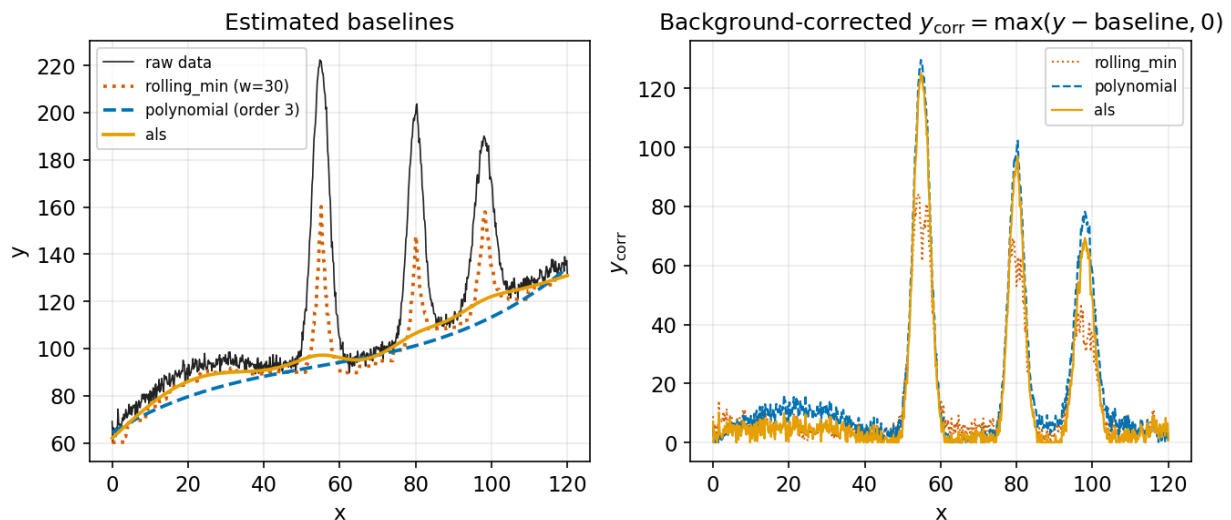


Abbildung 3. Die drei Basislinien-Methoden auf einem gekrümmten, verrauschten Untergrund. Links: die geschätzten Basislinien — das rollende Minimum folgt der unteren Einhüllenden in Stufen, Polynom und ALS sind glatt. Rechts: das korrigierte Signal; ALS und das Polynom ebnen den Untergrund zwischen den Peaks sauberer ein.

- **Mindest-SNR** (`min_snr_init`): ein Kandidat wird nur akzeptiert, wenn seine Höhe mindestens das `min_snr_init`-fache des geschätzten Rauschens beträgt. Ergänzt die Prominenz um ein rauschbezogenes Kriterium.
- **Untergrundfenster** (`bg_window`, in Punkten): die Fensterbreite der Untergrundschätzung (rollendes Minimum bzw. die gewählte `baseline_method`), die *vor* der Suche abgezogen wird. Zu klein frisst echte Peaks an, zu groß lässt eine gekrümmte Basislinie stehen; 0 schaltet den Abzug ab. Derselbe Untergrund geht in den Fit und die Präsenzprüfung ein.

In der GUI kann man die Peaks auch einfach anklicken (Linksklick setzen, Rechtsklick entfernen), was oft schneller ist als das Feinjustieren der Regler.

Toleranz (`tolerance`). Das $\pm$ -Fenster, in dem ein Komponenten-Peak als „derselbe“ wie ein Referenz-Peak gilt. Größer als die typische Schwerpunktsverschiebung zwischen Spektren wählen, aber kleiner als der Peak-Abstand.

Positionsverfeinerung (`refine`). Standardmäßig aus. Aktivieren, wenn die Positionen leicht daneben liegen (z. B. eine kleine Kalibrierverschiebung): die Zentren werden dann innerhalb von `refine_window` verfeinert, während die Amplituden nichtnegativ bleiben (Abbildung 4). Aus lassen, wenn die Positionen bekannt und verlässlich sind.

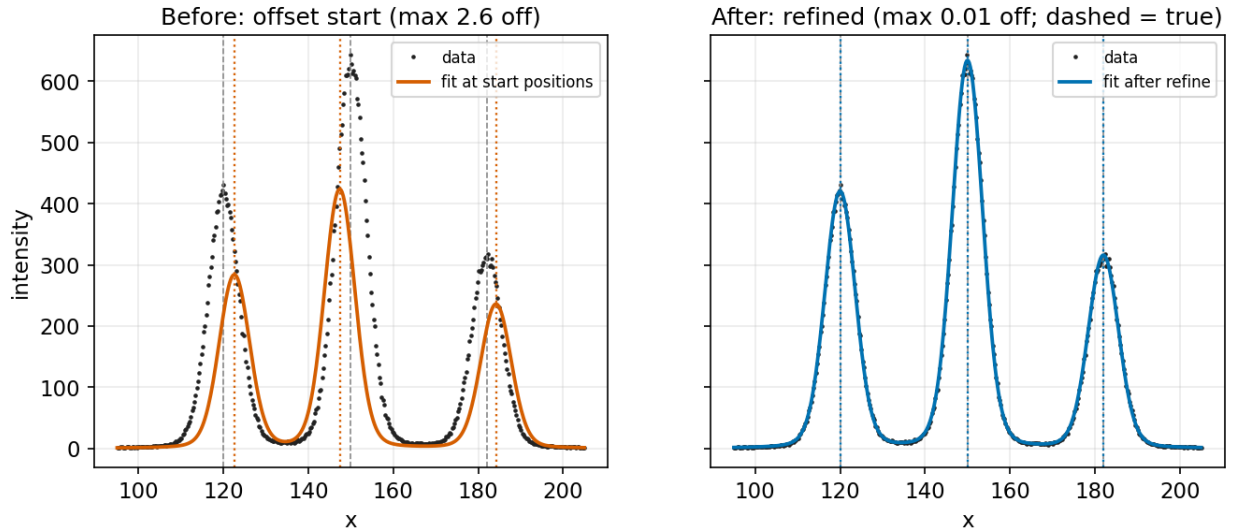


Abbildung 4. Optionale Positionsverfeinerung. Links: ein Fit mit absichtlich versetzten Startpositionen (gepunktet) verfehlt die wahren Zentren (grau gestrichelt). Rechts: mit `refine = true` werden die Zentren bis auf einen kleinen Bruchteil eines Punktes zurückgewonnen.

Breitenverfeinerung (`refine_widths`). Standardmäßig aus; sonst bleiben die Breiten für alle Peaks auf der instrumentellen Auflösung. Aktivieren, wenn eine Bande wirklich breiter als das Instrument ist (z. B. unauflöste Aufspaltung oder inhomogene Verbreiterung). Dann wird eine Breite pro Peak verfeinert, beschränkt zwischen `width_min_factor` und `width_max_factor` mal der instrumentellen Breite; `width_mode` wählt, ob der ganze Voigt ("`fwhm`") oder nur der Gauß-Anteil ("`sigma`") breiter wird (Abbildung 5). Die Obergrenze eng halten: zusätzliche freie Breiten senken das reduzierte χ^2_ν immer, und bei überlappenden oder schwachen Peaks tauschen Breite und Amplitude — ein kleineres χ^2_ν ist also für sich genommen kein Beleg für ein besseres Modell.

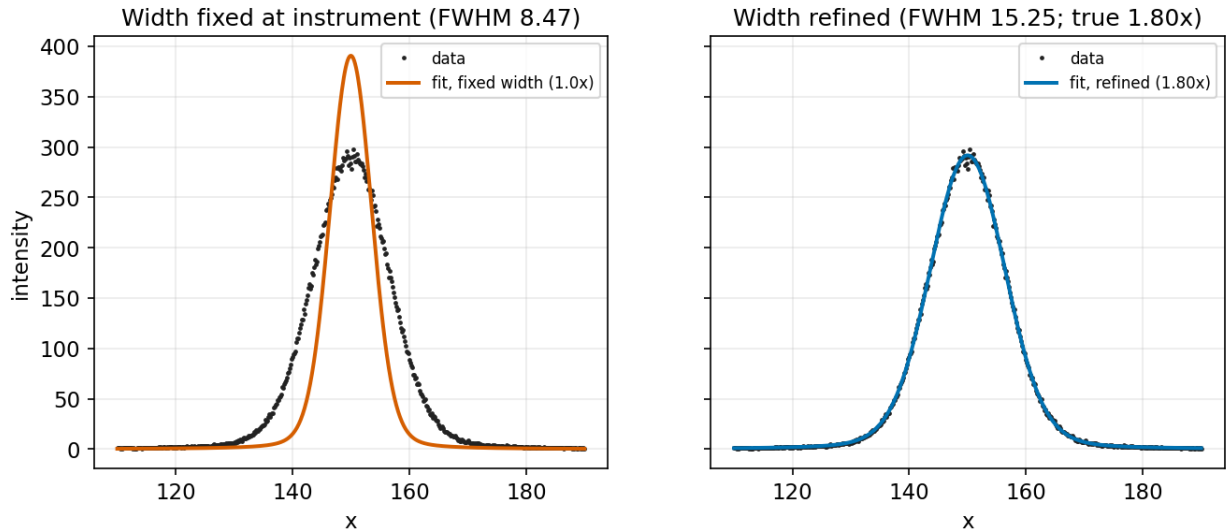


Abbildung 5. Optionale Breitenverfeinerung an einem Peak, der wirklich breiter als das Instrument ist. Links: mit fester instrumenteller Breite erreicht der Fit die Daten nicht. Rechts: mit `refine_widths = true` wird die eine Breite pro Peak zurückgewonnen (hier ein Faktor nahe der wahren Verbreiterung), begrenzt durch `width_min_factor..width_max_factor`.

8 Ausgabedateien

Alle Ergebnisse werden neben der Referenzdatei abgelegt; ein etwaiger Suchbereich erscheint im Dateinamen (z. B. `_90_200`).

Datei	Inhalt
<code>*_fit.png</code>	Fit mit Einzelkomponenten und Residuenpanel
<code>*_peaks.png</code>	je Komponente eine markierte Abbildung (grün = vorhanden, rot = fehlt)
<code>*_results.xlsx</code>	Excel mit sechs Blättern: Intensities, Positions, Presence, Fit_Data, Components_Raw, Parameters (Components_Raw nur, wenn Komponenten ausgewertet werden)
<code>*_results.csv</code>	eine reine 7-bit-ASCII-CSV mit fünf benannten Sektionen (Intensities, Positions, Presence, Fit_Data, Components_Raw) und dem vollständigen Parameterblock als Kopfzeile

Das Excel-Blatt **Presence** ist die Kernaussage: eine Binärmatrix, die für jeden Peak und jede Komponente 1 (grün, vorhanden) oder 0 (rot, fehlt) zeigt. Das Blatt **Parameters** dokumentiert jeden Lauf vollständig, sodass er reproduzierbar bleibt.

Die Plots lesen. Abbildungen 6 und 7 beschriften die beiden Plottypen. Im Fit-Plot ist die schwarze Kurve die Summe, die farbigen Kurven sind die einzelnen Peaks (jede Fläche ist die gefittete Amplitude), und das untere Feld zeigt die Residuen — ein guter Fit lässt sie strukturlos um null streuen, und der Titel nennt χ^2_ν , R^2 und die Zahl der aktiven Peaks. In einem Komponenten-Plot bedeutet eine grüne gestrichelte Linie mit Marker, dass der Peak vorhanden ist (1); eine rote Linie bedeutet abwesend (0). Die Entscheidung fällt innerhalb des $\pm\text{tolerance}$ -Fensters (schattiert) anhand der SNR-Schwelle.

9 Fehlerbehebung & FAQ

Die GUI startet nicht. Auf einem Rechner ohne Display oder ohne Tk die Kommandozeile nutzen: `peakcheck -config job.toml -no-gui`. Unter Debian/Ubuntu Tk per `sudo apt install python3-tk` installieren.

„-no-gui needs a config file“. Der Headless-Modus braucht eine TOML (oder zumindest `-reference`). Eine Vorlage schreiben mit `peakcheck -write-template job.toml`.

Es werden keine Peaks gefunden. `prominence_init` und/oder `min_snr_init` senken, das Suchfenster `x_min/x_max` prüfen oder die Peaks in der GUI anklicken. Die *x conversion* prüfen, damit die Positionen in den erwarteten Einheiten liegen.

Reduziertes χ^2_ν liegt weit über 1. Das Modell mit fester Form wird strapaziert: die Breiten `wg/wl` passen vermutlich nicht zu den Daten, oder ein Peak fehlt. Fehlenden Peak ergänzen oder Breiten korrigieren; die Amplituden bleiben der beste nichtnegative Fit.

Gesetzte Peaks verschwinden nach dem Fit (aktive Anzahl sinkt) oder der Fit ist ein breiter Hügel

Das Profil ist viel breiter als die Peaks der Daten; die überlappenden Profile werden redundant, und der nichtnegative Fit setzt einige Amplituden auf null. Fast immer sind `wg/wl` auf der falschen Skala: z. B. instrumentelle Werte in cm^{-1} , während die Daten noch in meV vorliegen, weil die *x-Umrechnung* aus ist. Umrechnung setzen (oder die passende Config laden) oder `wg/wl` grob auf die Halbwertsbreite der Peaks stellen. PeakCheck gibt in diesem Fall einen Hinweis aus. Die Marker- und die Fit-Beschriftung zeigen stets dieselben Positionen; nur ihre

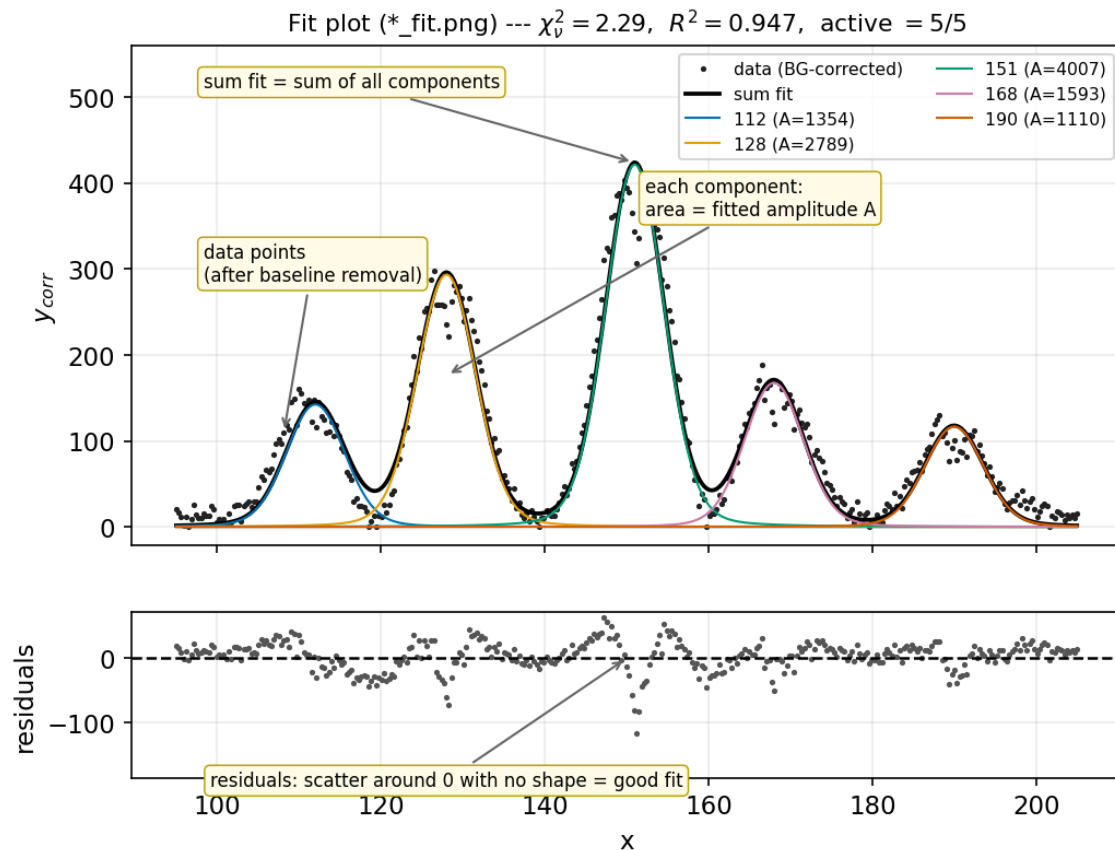


Abbildung 6. So liest man `*_fit.png`: die Daten nach Untergrundabzug, der summierte Fit, die einzelnen Komponenten (Fläche = Amplitude), die Gütezusammenfassung im Titel und das Residuenfeld darunter.

angezeigte Genauigkeit stimmt jetzt überein (eine Nachkommastelle), ein Peak „wandert“ also nicht zwischen Setzen und Fitten.

Ein erwarteter Peak gilt als „abwesend“. `tolerance` erhöhen, wenn der Komponenten-Peak verschoben ist, oder `snr_thresh` für eine verrauschte Komponente senken. Prüfen, ob `presence_baseline_c` der Absicht entspricht (Höhe über Basislinie vs. Rohsignal).

Die gemeldeten Fehler wirken falsch. Mit echter Fehlerspalte liefert der Modus `statistical` wahre 1σ ; ohne sie werden Fehler mit $\sqrt{\chi^2_p}$ skaliert. Für Zählraten über viele Größenordnungen eine Fehlerspalte angeben. Das Supplement zeigt eine Monte-Carlo-Prüfung des Fehlerbudgets.

Die Datei lädt nicht. PeakCheck liest zwei- oder dreispaltigen Whitespace- oder CSV-Text und ignoriert Kommentarzeilen, die mit `#`, `!` oder `%` beginnen; nicht-finite Zeilen werden verworfen. Trennzeichen und Fehlerspalten-Einstellung prüfen.

Kann ich einen Lauf exakt reproduzieren? Ja — die Config speichern (*Save config*, Strg+S) oder den Parameterblock aus dem CSV-Kopf bzw. dem Excel-Blatt `Parameters` wiederverwenden und mit `-config` erneut ausführen.

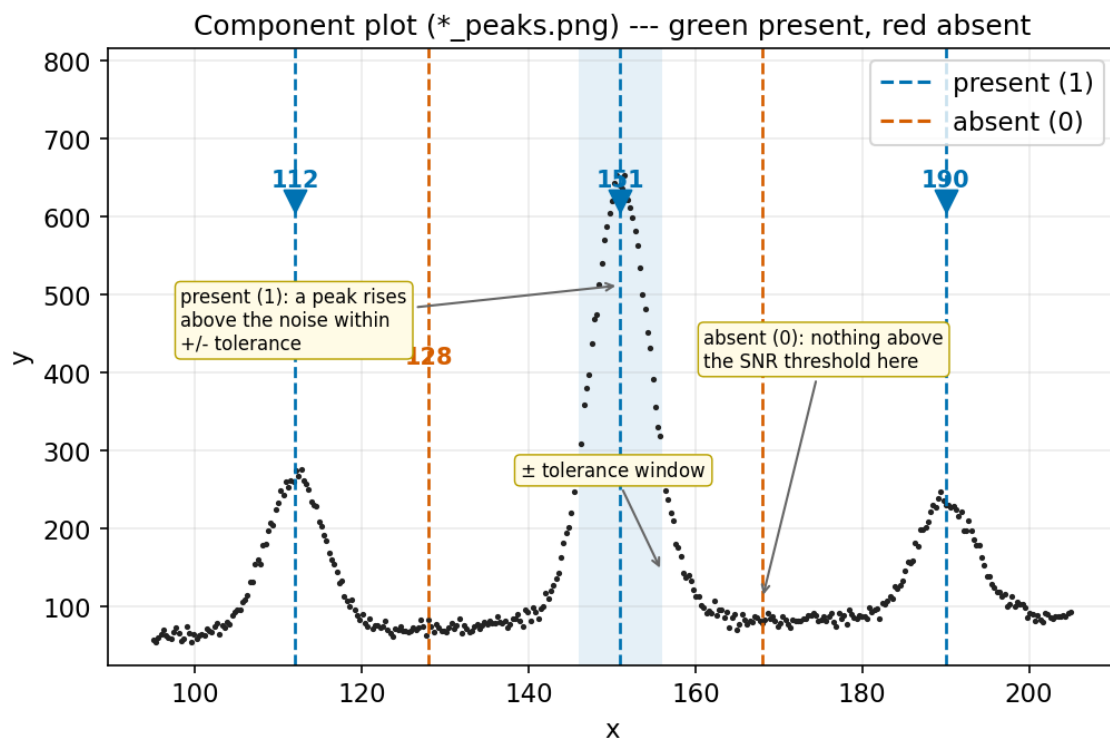


Abbildung 7. So liest man *_peaks.png: ein grüner Marker/eine grüne Linie markiert einen vorhandenen Peak (1), eine rote Linie einen abwesenden (0); die Entscheidung fällt innerhalb des $\pm$ tolerance-Fensters (schattiert).